

2 Проектно-конструкторский раздел

2.1 Выбор средств разработки

2.1.1 Выбор языка программирования для разработки клиента и сервера

Для реализации клиентской части веб-приложения в качестве основного языка программирования выбран TypeScript. Данный выбор обусловлен несколькими ключевыми факторами: TypeScript является строго типизированным надмножеством JavaScript, что позволяет выявлять ошибки на этапе компиляции и повышает надёжность кода, широкая экосистема инструментов разработки для TypeScript обеспечивает эффективный процесс создания крупных приложений, поддержка JSX-синтаксиса позволяет комфортно работать с библиотекой React.

В качестве основной библиотеки для построения пользовательского интерфейса выбран React. Данный фреймворк обеспечивает декларативный подход к разработке, компонентную архитектуру и эффективный механизм обновления DOM через виртуальное представление. Использование TypeScript в сочетании с React позволяет создавать типобезопасные компоненты с поддержкой интеллектуального автозаполнения в средах разработки.

Для сборки проекта применяется инструмент Vite, который обеспечивает быстрый Hot Module Replacement (HMR) и оптимизированный билд для производственной среды.

2.1.2 Выбор библиотек и фреймворков

При создании клиентской части приложения были проанализированы доступные инструменты для маршрутизации навигации, работы с HTTP-запросами и визуального оформления интерфейса. Для организации клиентской маршрутизации выбран react-router-dom, предоставляющий декларативный способ определения путей и управления состоянием навигации без перезагрузки страницы. Данный пакет обеспечивает историю переходов между экранами приложения и поддерживает вложенные маршруты, что необходимо для структуры веб-приложения с множеством функциональных разделов.

Для осуществления HTTP-запросов к серверной части выбран axios. Библиотека предоставляет promise-based API для выполнения сетевых запросов, поддерживает автоматическую десериализацию JSON ответов, настройку заголовков авторизации и возможность создания интерсепторов для перехвата запросов и ответов. Эти возможности особенно важны для приложения, работающего с внешними системами поиска уязвимостей.

Для визуального оформления интерфейса применена библиотека Tailwind CSS. Данный фреймворк предлагает utility-first подход к стилизации, позволяя определять внешний вид элементов непосредственно в HTML-разметке через классовые утилиты. Такой подход ускоряет разработку компонентов и обеспечивает консистентность дизайна во всём приложении.

Для работы с иконками используется пакет react-feather, предоставляющий набор SVG-иконок в виде React-компонентов. Данный выбор обусловлен лёгкостью интеграции, минимальным размером пакета и единообразием стиля визуальных элементов.

Генерация отчётов о результатах поиска уязвимостей реализована с помощью библиотеки pdfmake. Данная библиотека позволяет создавать PDF-документы программно на стороне клиента, формируя структурированные документы с таблицами результатов и текстовыми описаниями найденных уязвимостей.

2.1.3 Выбор среды разработки

В качестве основной среды разработки для клиентской части выбран VS Code. Данный выбор обусловлен широкими возможностями по настройке и расширению функциональности, а также поддержкой множества языков программирования и фреймворков через систему расширений. Критически важными расширениями в процессе разработки стали ESLint и Prettier — первое обеспечивает статический анализ кода на соответствие стандартам качества, второе автоматизирует форматирование исходного текста согласно единому стилю оформления. Подобный подход позволяет выявить потенциальные проблемы ещё до компиляции проекта и поддерживать единообразие кода при работе с проектом.

2.1.4 Выбор операционной среды развертывания

Для развёртывания серверной части выбран дистрибутив Ubuntu Linux, предлагающий надёжную и защищённую операционную систему с длительным сроком поддержки стабильных релизов. Данный выбор обусловлен преимуществами перед Windows Server в контексте веб-разработки: открытые исходные коды большинства компонентов экосистемы, оптимизированное управление ресурсами при высоких нагрузках и богатый выбор инструментов контейнеризации.

Для организации оркестрации сервисов применяется Docker с использованием docker-compose для управления многоконтейнерными приложениями. Такое решение позволяет изолировать зависимости бэкенда (Python 3.x, FastAPI, SQLAlchemy), базы данных PostgreSQL от операционной системы хоста и обеспечить воспроизводимость среды развёртывания на любых машинах. Контейнеры сервера и клиента запускаются в едином docker-compose.yml файле, что упрощает процесс деплоя.

2.1.5 Выбор СУБД

В качестве системы управления базами данных выбрана PostgreSQL 17 — реляционная СУБД с открытым исходным кодом, признанная одной из наиболее надёжных решений для хранения структурированных данных. Данный выбор обусловлен поддержкой сложных SQL-запросов, транзакционной целостностью и расширенными возможностями индексирования, необходимыми для эффективного поиска уязвимостей по множеству параметров одновременно.

Для работы с ORM применена библиотека SQLAlchemy, объединяющая возможности Pydantic для валидации данных и SQLAlchemy для управления связями между таблицами. Миграции схемы базы данных осуществляются через Alembic — независимый инструмент управления изменениями структуры БД, позволяющий версионировать эволюцию схем вместе с кодовой базой приложения.

2.2 Проектирование архитектуры веб-приложения

Веб-приложение Vulnerability Checker построено на основе клиент-серверной архитектуры с модульным разделением ответственности между компонентами системы. Веб-клиент реализован как SPA (Single Page Application) на React 19 и взаимодействует с серверным API посредством RESTful HTTP запросов. Авторизация осуществляется через JWT токены, а для внешних вызовов предусмотрен механизм X-API-Key аутентификации.

Клиентское приложение (SPA) обрабатывает пользовательский ввод и отправляет HTTP запросы к серверному API. Веб-интерфейс использует React Router DOM для навигации между страницами: Landing, Login, Register, Scan (проверка пакетов) и Profile.

Серверная часть представлена модулем FastAPI — современным асинхронным фреймворком для создания высокопроизводительных API на языке Python. Данный выбор обусловлен поддержкой async/await синтаксиса, встроенной валидацией входных данных через Pydantic и автоматической генерацией документации OpenAPI (Swagger UI). Сервер выступает

посредником между клиентом и базой данных PostgreSQL, реализуя бизнес-логику сравнения версий пакетов с уязвимыми версиями из репозитория.

Архитектурная схема взаимодействия компонентов включает следующие уровни:

- веб-клиент на React обрабатывает пользовательский ввод и отправляет HTTP запросы к серверному API;
- слой FastAPI маршрутизирует запросы к соответствующим сервисам и выполняет валидацию данных;
- уровень бизнес-логики в модуле `scan_service.py` сравнивает версии пакетов с уязвимыми версиями
- из базы данных; слой доступа к данным через SQLAlchemy обеспечивает взаимодействие с PostgreSQL.

Безопасность передачи данных обеспечивается применением JWT токенов (access token сроком действия 30 минут и refresh token на 7 дней) для авторизации пользователей в веб-клиенте, а также механизмом X-API-Key аутентификации для внешних вызовов API. API Key автоматически генерируется при создании аккаунта пользователя (`uuid.uuid4().hex`).

2.3 Проектирование интерфейса

При открытии веб-приложения выводится стартовая страница (Landing) с общей информацией о сервисе проверки уязвимостей. На странице доступны ссылки для входа в систему или регистрации нового аккаунта.

Страница логина содержит форму ввода email и пароля для авторизации существующего пользователя. Ниже формы расположена кнопка «Войти», после нажатия на которую отправляется запрос к серверу и при успешной аутентификации пользователь перенаправляется на главную страницу сканирования.

Страница регистрации содержит поля для ввода email, имени, фамилии, пароля и опциональных данных (отчество, телефон). Ниже формы находится кнопка «Зарегистрироваться», после нажатия на которую создаётся аккаунт пользователя и автоматически генерируется API ключ для проверки пакетов. После успешной регистрации пользователь также попадает на страницу сканирования.

Главная страница сканирования содержит заголовок с описанием сервиса и карточку загрузки файлов с зоной drag-and-drop. После загрузки файла зависимостей появляется кнопка «Сканировать на уязвимости», после нажатия на которую инициируется проверка загруженного файла. При процессе сканирования отображается прогресс-бар с анимацией и текстовым статусом «Идет сканирование файла...».

Экран отображения результатов сканирования выводит список проверенных библиотек с указанием статуса проверки. Для каждого пакета, имеющего уязвимости, выводятся карточки CVE с информацией об идентификаторе угрозы, уровне опасности (критичный, высокий, средний, низкий) и балле CVSS. Безопасные пакеты отображаются без дополнительных деталей.

На каждой странице расположена панель навигации в верхней части экрана со ссылками на основные разделы: сканирование пакетов, профиль пользователя и выход из системы. Панель автоматически обновляет доступные пункты в зависимости от текущего состояния авторизации.

Страница профиля содержит персональные данные пользователя (email, имя, телефон) и список его API ключей, которые используются для аутентификации при вызове серверного API. Интерфейс позволяет скопировать любой ключ для последующего использования.

Результаты сканирования можно экспортировать в формате PDF отчёта или текстового файла через специальные кнопки на странице с результатами. PDF-отчёт содержит титульную страницу со статистикой, таблицу всех обнаруженных уязвимостей и списки проверенных пакетов с разделением на уязвимые и безопасные.

2.4 Проектирование базы данных

Важным этапом в создании программного обеспечения является проектирование базы данных. Оно задаёт структуру и порядок хранения данных.

2.4.1 Концептуальная схема

Концептуальная модель представляет собой высокоуровневое описание предметной области. Она определяет ключевые сущности системы и взаимосвязи между ними.

Основные сущности системы:

- Пользователь (User) – содержит учетные данные, личную информацию и API ключи зарегистрированных пользователей системы;
- API ключ (Api key) – служит для аутентификации внешних вызовов серверного API;
- Пакет (Package) – хранит данные о библиотеках и фреймворках, доступных в официальном репозитории;
- Версия пакета (Package version) – представляет собой конкретную версию библиотеки с указанием номера релиза;
- Уязвимость (Vulnerability) – содержит информацию о известных CVE для сравнения версий пакетов;
- Компания (Company) – организует пользователей в рамках корпоративной структуры.

Связь между сущностями отображена на рисунке 10.

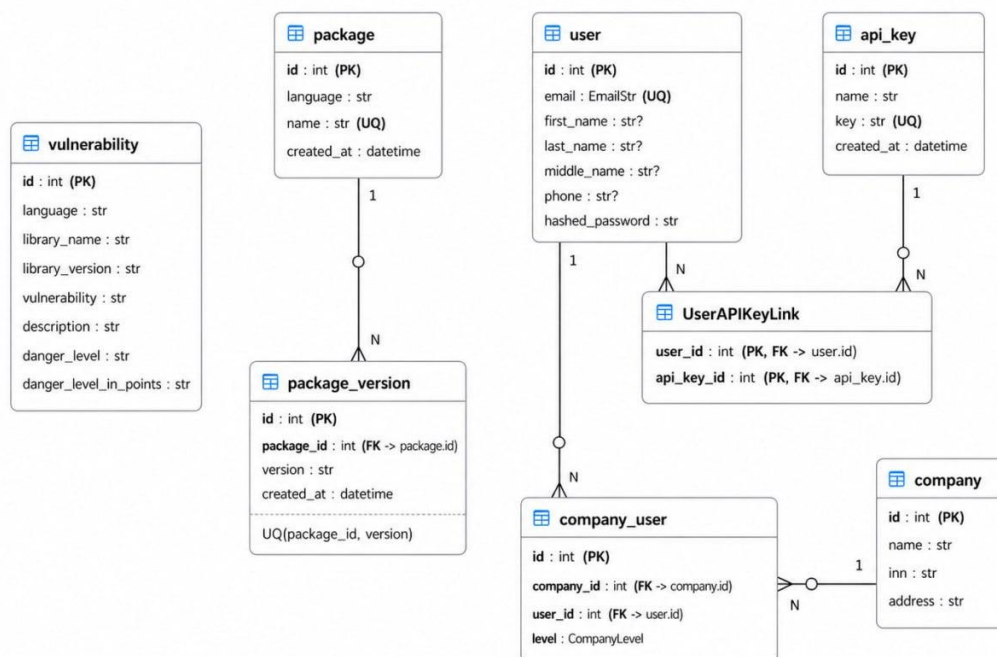


Рисунок 10 – Схема базы данных

2.4.2 Логическая схема

Логическая модель преобразует концептуальное представление в формальное описание таблиц с указанием атрибутов, типов данных и ключевых полей.

Описание таблиц и их структура представлены в таблицах 9-15.

Информация о пользователях системы содержится в таблице User:

Имя поля	Описание поля	Ключ	Тип данных
id	Идентификатор пользователя	Первичный	INTEGER
email	Электронная почта	Уникальный, NOT NULL	TEXT
first_name	Имя	—	TEXT
last_name	Фамилия	—	TEXT
middle_name	Отчество	—	TEXT
phone	Телефон	—	TEXT
hashed_password	Захешированный пароль (bcrypt/scrypt)	NOT NULL	TEXT

Информация об API ключах пользователей хранится в таблице Api key:

Имя поля	Описание поля	Ключ	Тип данных
id	Идентификатор ключа	Первичный	INTEGER
name	Название ключа	—	TEXT
key	Случайно сгенерированный ключ (uuid)	Уникальный, NOT NULL	TEXT
created_at	Дата создания	—	TIMESTAMP

Связь между пользователями и ключами реализуется через таблицу UserAPIKeyLink:

Имя поля	Описание поля	Ключ	Тип данных
user_id	Идентификатор пользователя	Составной (PK), Внешний	INTEGER
api_key_id	Идентификатор API ключа	Составной (PK), Внешний	INTEGER

Данные о пакетах хранятся в таблице Package:

Имя поля	Описание поля	Ключ	Тип данных
id	Идентификатор пакета	Первичный	INTEGER
language	Язык программирования	—	TEXT
name	Название пакета	Уникальный, NOT NULL	TEXT
created_at	Дата добавления в базу	—	TIMESTAMP

Версии пакетов хранятся в таблице Package version:

Имя поля	Описание поля	Ключ	Тип данных
id	Идентификатор версии	Первичный	INTEGER
package_id	Идентификатор пакета	Внешний, NOT NULL	INTEGER
version	Номер версии	Составной (с package_id)	TEXT
created_at	Дата добавления	—	TIMESTAMP

База данных уязвимостей хранится в таблице Vulnerability:

Имя поля	Описание поля	Ключ	Тип данных
id	Идентификатор уязвимости	Первичный	INTEGER
language	Язык программирования	—	TEXT
library_name	Название библиотеки	—	TEXT
library_version	Уязвимая версия	—	TEXT
vulnerability	Идентификатор CVE	—	TEXT
description	Описание уязвимости	—	TEXT
danger_level	Уровень опасности	—	TEXT
danger_level_in_points	Балл опасности (CVSS)	—	TEXT

Информация о компаниях хранится в таблице Company:

Имя поля	Описание поля	Ключ	Тип данных
id	Идентификатор компании	Первичный	INTEGER
name	Название компании	—	TEXT
inn	ИНН организации	—	TEXT
address	Адрес регистрации	—	TEXT

Связь между компаниями и пользователями реализуется через таблицу Company user:

Имя поля	Описание поля	Ключ	Тип данных
id	Идентификатор связи	Первичный	INTEGER
company_id	Идентификатор компании	Внешний, NOT NULL	INTEGER
user_id	Идентификатор пользователя	Внешний, NOT NULL	INTEGER
level	Уровень доступа (admin/employee)	—	TEXT

2.4.3 Физическая схема

2.5